UNIVERSIDAD NACIONAL DEL ALTIPLANO

FACULTAD DE INGENIERÍA MECÁNICA ELÉCTRICA, ELECTRÓNICA Y SISTEMAS

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



INFORME DEL PROYECTO:

SISTEMA DE GESTIÓN INTEGRAL PARA TIENDA DE CELULARES

SECCIÓN: GRUPO A

CURSO: PROGRAMACIÓN ORIENTADA A OBJETOS I

INTEGRANTES:

HUANCA CHOQUEHUANCA JUAN JOSÉ

HUALLPA AQUINO FREDDY FRANKLIN

MENDOZA TOLA JOE ALDAIR

DOCENTE: ACEITUNO ROJO MIGUEL ROMILIO

PUNO – PERÚ

2026

ÍNDICE

1. RESUMEN 4
2. PLANTEAMIENTO DEL PROBLEMA Y OBJETIVOS 5
2.1. Identificación del Problema: Necesidad de un Sistema que Registre Compras 5
2.1.1. Necesidades del Negocio (Lógica Comercial) 5
2.1.2. Necesidades Técnicas (Ingeniería de Software) 6
2.1.3. Necesidad Académica 6
2.2. Objetivo General 6
2.3. Objetivos Específicos 7
3. MARCO TEÓRICO 7
3.1. Abstracción 7
3.2. Encapsulamiento 8
3.3. Herencia 8
3.4. Polimorfismo 9
3.5. Gestión Dinámica de Memoria y Asociación de Objetos 9
3.6. Síntesis Arquitectónica del Sistema 10
3.7. Modelado de la Jerarquía de Personas 10
4. DISEÑO UML Y DIAGRAMAS POR MÓDULOS 11
4.1. Arquitectura Taxonómica General del Sistema 12
4.2. Desglose de las Cinco Hojas del Diagrama de Clases UML 12
4.3. Especificación Detallada de Bloques, Atributos y Relaciones 14
5. IMPLEMENTACIÓN Y ARQUITECTURA DEL CÓDIGO 16
5.1. Transición del Diseño UML al Código, Encapsulamiento de Productos y Gestión de Memoria 16
5.2. Módulo Transaccional, Filtro Financiero, Administración de Nómina y Sincronización en RAM 17
5.3. Código Fuente Completo del Sistema 19
6. PRUEBAS Y RESULTADOS DE EJECUCIÓN 28
6.1. Caso de Prueba 1: Menú Principal, Autenticación Polimórfica y Catálogo de Celulares 29
6.2. Caso de Prueba 2: Validación del Filtro Financiero ante Saldo Insuficiente 30
6.3. Caso de Prueba 3: Compra Exitosa, Emisión de Boleta de Venta e Historial 31

6.4. Caso de Prueba 4: Gestión Administrativa, Balance de Caja y Pago de Nómina 32
6.5. Caso de Prueba 5: Actualización de Stock en Almacén y Cierre del Sistema..... 33
7. CONCLUSIONES Y RECOMENDACIONES 35
7.1. Conclusiones..... 35
7.2. Recomendaciones 35
8. BIBLIOGRAFÍA 37
Anexo 1..... 38

1. RESUMEN

El presente informe técnico documenta el diseño arquitectónico en UML, la implementación en lenguaje C++ y la validación funcional de un sistema orientado a objetos creado para resolver la necesidad de contar con un software que registre compras y automatice las operaciones de una tienda de celulares. El sistema desarrollado satisface requerimientos en dos grandes frentes complementarios: **Necesidades del Negocio (Lógica Comercial)** mediante la automatización de ventas y cálculo de comprobantes, el control financiero por medio del Filtro Financiero, la gestión diferenciada de personal y roles, el control de inventario detallado y la logística de entrega y garantía. Las **Necesidades Técnicas (Ingeniería de Software)** garantizan la robustez ante errores de entrada en consola, eficiencia y seguridad en el manejo de memoria con punteros y destructores virtuales, escalabilidad mediante herencia y polimorfismo, y una interfaz de usuario clara manteniendo plena fidelidad con el diseño previo de cinco hojas en UML.

Palabras clave: *Registro de compras, automatización de ventas, control financiero, gestión de inventario, diseño UML, C++, arquitectura orientada a objetos.*

2. PLANTEAMIENTO DEL PROBLEMA Y OBJETIVOS

2.1. Identificación del Problema: Necesidad de un Sistema que Registre Compras

El problema principal que origina el presente proyecto es la necesidad de contar con un sistema que registre compras y gestione de forma estructurada las operaciones de una tienda de celulares. El código y la arquitectura desarrollados satisfacen necesidades en dos grandes frentes los cuales son: Necesidades del Negocio (Lógica Comercial) y Necesidades Técnicas (Ingeniería de Software) además de cumplir con la fidelidad académica al diseño arquitectónico UML.

2.1.1. Necesidades del Negocio (Lógica Comercial)

El sistema resuelve los problemas diarios de una tienda de celulares en cinco aspectos clave:

- **Automatización de Ventas:** Sustituye el proceso manual de calcular totales y emitir comprobantes. El carrito (`Venta` y `LineaVenta`, junto con el flujo transaccional en la clase `Tienda`) calcula el subtotal y el importe total automáticamente.
- **Control Financiero:** Evita que se realicen ventas a pérdida o a clientes sin fondos. El **Filtro Financiero** verifica el presupuesto del cliente antes de autorizar la compra, calculando y mostrando exactamente cuánto dinero se requiere frente al saldo disponible si la transacción es rechazada.
- **Gestión de Personal y Roles:** Satisface la necesidad de separar responsabilidades. Los cajeros o vendedores no pueden alterar las finanzas de la tienda, y solo el `Administrador` puede pagar la nómina y gestionar los servicios operativos (luz, internet).
- **Control de Inventario Detallado:** Permite a los clientes y cajeros visualizar no solo el precio y stock, sino también la ficha técnica exacta de los equipos (RAM, ROM, batería, cámara, pantalla) para tomar mejores decisiones de compra.
- **Logística de Entrega:** Permite diferenciar si el cliente recogerá el producto en tienda (`EntregaLocal`) o si se lo enviarán a su domicilio (`EntregaDomicilio`), además de registrar la garantía del fabricante (`Garantia`).

2.1.2. Necesidades Técnicas (Ingeniería de Software)

El código satisface los requerimientos de un buen desarrollo de software mediante cuatro propiedades fundamentales:

- **Robustez ante Errores:** Evita que el sistema colapse si el usuario ingresa letras en lugar de números gracias a la validación de entrada numérica con `cin.fail()`, `cin.clear()` y `cin.ignore()`.
- **Eficiencia y Seguridad de Memoria:** Satisface la necesidad de un software estable al utilizar punteros verificados y liberar la memoria correctamente con destructores virtuales (`virtual ~Persona()`), previniendo fugas de memoria (*memory leaks*) y errores de segmentación.
- **Escalabilidad y Mantenimiento:** Gracias a la herencia, si mañana la tienda quiere vender *Tablets* o *Smartwatches*, solo se crea una nueva clase hija de `Producto` sin modificar el resto del sistema. Si se requiere agregar un nuevo rol (por ejemplo, *Supervisor*), solo se agrega al enumerado o jerarquía de `Rol`.
- **Interfaz de Usuario (UX en consola):** Satisface la necesidad de un software amigable. El formateo tabular con `<iomanip>` y la capacidad de integrar colores ANSI, barras de progreso (`<thread>`) y secuencias visuales hacen que la interacción sea ordenada y visualmente intuitiva.

2.1.3. Necesidad Académica

Fidelidad al UML: Satisface al 100 % el diseño previo en las 5 hojas elaboradas en Excalidraw, demostrando planificación previa a la codificación y respeto por los diagramas de clases y sus relaciones (1 a 1, 1 a muchos). En conclusión, el sistema satisface la necesidad de digitalizar y automatizar la operación de una tienda, protegiendo la integridad financiera del negocio mediante validaciones lógicas, y garantizando un software seguro en el manejo de memoria y altamente escalable gracias a la arquitectura orientada a objetos.

2.2. Objetivo General

Desarrollar un sistema que registre compras y automatice la operación comercial, de inventario y financiera de una tienda de celulares en C++, protegiendo la integridad financiera del

negocio mediante validaciones lógicas y garantizando un manejo seguro de memoria y alta escalabilidad.

2.3. Objetivos Específicos

- Modelar el dominio del sistema en cinco hojas de diagramas de clases UML (Personas y Enumeraciones, Productos e Inventario, Logística y Servicios, Transaccional, y Administración y Finanzas), estableciendo sus relaciones de herencia y asociación (1 a 1 y 1 a muchos).
- Automatizar el registro de compras y la emisión de comprobantes validando las existencias y el presupuesto del comprador mediante el **Filtro Financiero**.
- Separar las responsabilidades operativas del personal mediante la jerarquía polimórfica de `Persona`, `Cliente` y `Empleado`, restringiendo el pago de nómina y el control de caja al rol de `Administrador`.
- Garantizar la robustez ante errores de entrada en consola (`cin.clear()`, `cin.ignore()`) y la seguridad de memoria mediante punteros verificados y destructores virtuales.

3. MARCO TEÓRICO

El desarrollo del sistema se fundamenta en el paradigma de la Programación Orientada a Objetos, utilizando C++ como lenguaje de implementación debido a su alto rendimiento, tipado estático fuerte y control directo sobre los recursos de memoria (Stroustrup, 2013). A continuación, se exponen los fundamentos teóricos que sustentan la arquitectura construida.

3.1. Abstracción

La abstracción consiste en simplificar una realidad compleja extrayendo únicamente las características y comportamientos esenciales de una entidad para representarla dentro del modelo computacional. Como afirma Booch et al. (2007), "una abstracción denota las características esenciales de un objeto que lo distinguen de todos los demás tipos de objetos y proporciona límites conceptuales nítidamente definidos desde la perspectiva del observador" (p. 44). En el diseño de lenguajes como C++, este principio se materializa a través de clases abstractas que declaran al

menos una función virtual pura (`= 0`), impidiendo instanciar conceptos incompletos y obligando a las subclases a proporcionar una definición concreta (Stroustrup, 2013).

Asimismo, la abstracción de datos separa las propiedades lógicas de una entidad respecto a los detalles internos de su implementación física, logrando que los módulos del programa interactúen mediante contratos claros (Deitel y Deitel, 2014). En el modelado de sistemas comerciales, identificar adecuadamente las abstracciones clave del negocio como (`Persona`) y (`Producto`), reduce la complejidad algorítmica y facilita la evolución modular del software frente a nuevos requerimientos (Joyanes Aguilar, 2006).

3.2. Encapsulamiento

El encapsulamiento agrupa el estado interno de un objeto y las operaciones que lo manipulan dentro de una misma unidad lógica, restringiendo el acceso directo a sus atributos desde el exterior. De acuerdo con Deitel y Deitel (2014), "las clases encapsulan (es decir, envuelven) los atributos y las funciones miembro en unidades definidas por el programador, ocultando los detalles de implementación para evitar modificaciones accidentales o inválidas de los datos" (p. 388). Este ocultamiento de información (*information hiding*) complementa a la abstracción al proteger las invariantes de cada clase mediante especificadores de acceso restrictivos (`private`), asegurando que ningún módulo externo corrompa el estado interno de un objeto (Booch et al., 2007).

En C++, el uso riguroso de métodos selectores (*getters* calificados con `const`) y modificadores (*setters* con validaciones lógicas) garantiza que cualquier cambio de estado como descontar saldo de la `billetera`, actualizar la `caja` o reducir el `stock` cumpliendo previamente con las reglas del negocio (Ceballos Sierra, 2018). Además, el encapsulamiento reduce el acoplamiento entre los módulos del programa, permitiendo modificar la estructura interna de una clase sin alterar el resto del sistema mientras su interfaz pública permanezca intacta (Joyanes Aguilar, 2006).

3.3. Herencia

La herencia permite crear nuevas clases derivadas que reutilizan, extienden y especializan el comportamiento de una superclase existente. Como explica Ceballos Sierra (2018), "la herencia es el mecanismo mediante el cual una clase derivada hereda los miembros de una clase base, permitiendo reutilizar código ya probado y establecer una relación jerárquica del tipo 'es-un' entre

conceptos" (p. 312). Mediante la herencia pública (`class Derivada : public Base`), los conceptos específicos del dominio comparten una estructura común definida en la clase padre, evitando la duplicación de atributos de identificación y credenciales (Stroustrup, 2013).

Durante la construcción de objetos derivados, el constructor de la subclase invoca explícitamente al constructor de la clase base mediante la lista de inicialización, garantizando que la subestructura heredada quede correctamente inicializada antes de añadir los atributos propios (Deitel y Deitel, 2014). Desde el punto de vista de la ingeniería de software, organizar las entidades en jerarquías de generalización y especialización (como `Cliente` y `Empleado` a partir de `Persona`, o `Celular` y `Accesorios` a partir de `Producto`) refleja con fidelidad la taxonomía real de una organización comercial (Booch, Rumbaugh y Jacobson, 2005).

3.4. Polimorfismo

El polimorfismo es la capacidad de un objeto de responder dinámicamente según su tipo concreto a través de una interfaz compartida. En palabras de Stroustrup (2013), "obtener el comportamiento correcto a partir de una jerarquía de clases independientemente del tipo exacto del objeto utilizado, a través de una interfaz común de funciones virtuales, se denomina polimorfismo" (p. 64). El polimorfismo dinámico o de enlace tardío (*late binding*) se implementa en C++ mediante funciones declaradas con la palabra clave `virtual` en la clase base y redefinidas con el especificador `override` en las clases derivadas (Deitel y Deitel, 2014).

Este mecanismo permite diseñar algoritmos genéricos que operan sobre punteros o referencias de la superclase, delegando en la tabla de funciones virtuales (*vtable*) la selección del método específico en tiempo de ejecución (Gamma et al., 1995). Gracias al polimorfismo, el sistema puede ampliarse incorporando nuevas subclases de usuarios o de productos sin necesidad de reescribir la lógica de control existente (Joyanes Aguilar, 2006).

3.5. Gestión Dinámica de Memoria y Asociación de Objetos

El manejo eficiente de la memoria y la asociación estructural de objetos son cruciales en C++. Como sostienen Gamma et al. (1995), "la composición y asociación de objetos permiten construir funcionalidades complejas ensamblando objetos existentes en tiempo de ejecución, manteniendo cada clase encapsulada y enfocada en una única responsabilidad" (p. 20). En C++, toda clase base polimórfica debe declarar un destructor virtual (`virtual ~Persona() {}`) para

garantizar que, cuando se destruye un objeto derivado, se libere íntegramente la memoria de la subclase y se prevengan fugas de memoria (Stroustrup, 2013).

Asimismo, el empleo de punteros tipados (`Celular*`, `Empleado*`, `Cliente*`) verificados contra `nullptr` permite referenciar objetos existentes en memoria sin incurrir en copias costosas, validando de forma segura las búsquedas sin coincidencias (Deitel y Deitel, 2014). Complementariamente, el uso de contenedores secuenciales como `std::vector` administra automáticamente la reserva y liberación de memoria dinámica en el *heap* al crecer el inventario o la lista de clientes (Ceballos Sierra, 2018).

3.6. Síntesis Arquitectónica del Sistema

Como síntesis arquitectónica, el sistema de la tienda de celulares abstrae el ecosistema comercial en tres capas cohesivas que interactúan en memoria RAM: (1) la **Capa de Actores (Persona, Cliente, Empleado)**, donde la herencia y el polimorfismo unifican la identidad y autenticación mientras diferencian los privilegios operativos y financieros de cada usuario; (2) la **Capa de Bienes Comerciales (Producto / Celular)**, donde el encapsulamiento resguarda el valor económico y las existencias físicas frente a alteraciones no autorizadas; y (3) la **Capa Orquestadora y Transaccional (Tienda)**, que articula por asociación las colecciones dinámicas de objetos, aplica el **Filtro Financiero** antes de comprometer cualquier recurso y sincroniza en tiempo real el flujo de caja entre las compras de los clientes y el pago de planillas del personal.

3.7. Modelado de la Jerarquía de Personas

El módulo de actores humanos se estructura a partir de la clase base abstracta `Persona`, la cual encapsula bajo acceso `private` cuatro atributos fundamentales: `int id`, `string nombre`, `string correo` y `string contrasena`. Su lectura se expone mediante métodos selectores constantes (`getId() const`, `getNombre() const`, `getCorreo() const`), mientras que la verificación de identidad se delega al método `bool validarCredenciales(const string& user, const string& pass) const`. Al declarar el método virtual puro `virtual string obtenerRol() const = 0;`, se establece el contrato polimórfico que implementan sus dos subclases:

- **Clase Cliente (: public Persona):** Modela al comprador minorista incorporando los atributos privados `double billetera` y `vector<string> historialCompras`. Define los métodos `descontarSaldo(double monto)` y `agregarCompra(const string& descripcion)`, y sobrescribe `obtenerRol() const override` retornando `"CLIENTE"`.
- **Clase Empleado (: public Persona):** Modela al trabajador de la tienda mediante los atributos privados `string rol` (`"Administrador"` o `"Vendedor"`), `double sueldo` y la bandera `bool sueldoPagado`. Su implementación de `obtenerRol() const override` devuelve el cargo asignado, asegurando que solo el Administrador acceda a las finanzas de la tienda.

Tabla 1

Estructura de la Jerarquía Polimórfica de Personas

Clase	Categoría en la Jerarquía	Atributos Privados Propios	Métodos Principales y Polimorfismo
Persona	Clase Base Abstracta	id (int), nombre (string), correo (string), contrasena (string)	validarCredenciales(), virtual ~Persona(), virtual obtenerRol() const = 0
Cliente	Clase Derivada (: public Persona)	billetera (double), historialCompras (vector<string>)	descontarSaldo(), agregarCompra(), getHistorial(), obtenerRol() override
Empleado	Clase Derivada (: public Persona)	rol (string), sueldo (double), sueldoPagado (bool)	getSueldo(), isSueldoPagado(), marcarSueldoPagado(), obtenerRol() override

Nota. Síntesis de los atributos encapsulados y métodos polimórficos implementados en la jerarquía de actores del sistema.

4. DISEÑO UML Y DIAGRAMAS POR MÓDULOS

Previo a la fase de codificación, se elaboró el modelado conceptual y estructural del dominio empleando el Lenguaje Unificado de Modelado (UML). Como señalan Booch, Rumbaugh y Jacobson (2005), el diseño mediante diagramas de clases permite abstraer las

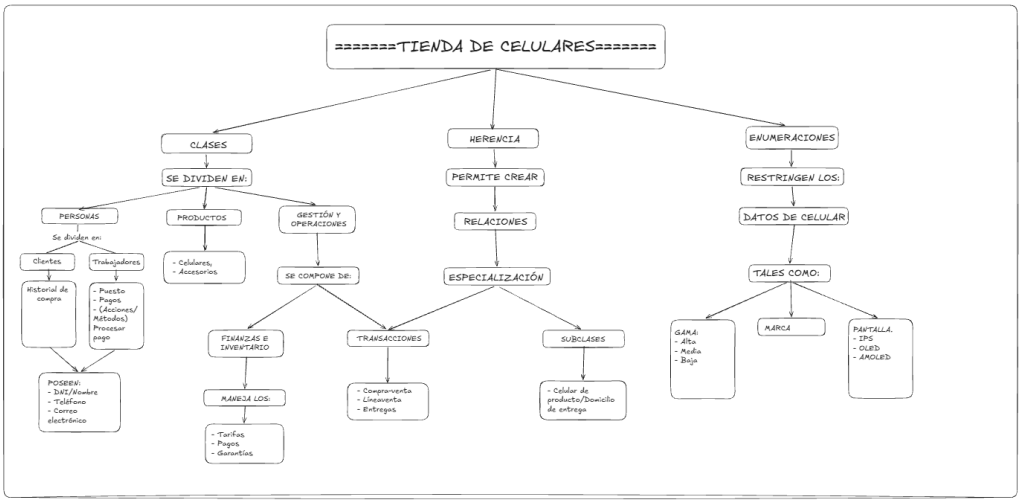
entidades del negocio, definir sus fronteras de encapsulamiento y especificar las relaciones de herencia y asociación con total claridad antes de su construcción en código.

4.1. Arquitectura Taxonómica General del Sistema

El diseño parte de un mapa taxonómico general que organiza el dominio de la tienda de celulares en tres grandes ramas estructurales: (1) las Clases del dominio (divididas en Personas, Productos y Gestión/Operaciones de finanzas, inventario y transacciones), (2) la Herencia como mecanismo generador de relaciones de especialización en transacciones y subclases operativas, y (3) las Enumeraciones que acotan los valores válidos para los datos técnicos del celular, tales como Gama (Alta, Media, Baja), Marca y tecnología de Pantalla (IPS, OLED, AMOLED).

Figura 1

Mapa Taxonómico General de Clases, Herencia y Enumeraciones del Sistema Tienda de Celulares



Nota. Diagrama conceptual general que estructura el sistema en tres ramas principales: Clases (Personas, Productos y Gestión/Operaciones), Herencia (relaciones de especialización en transacciones y subclases) y Enumeraciones de datos técnicos del celular.

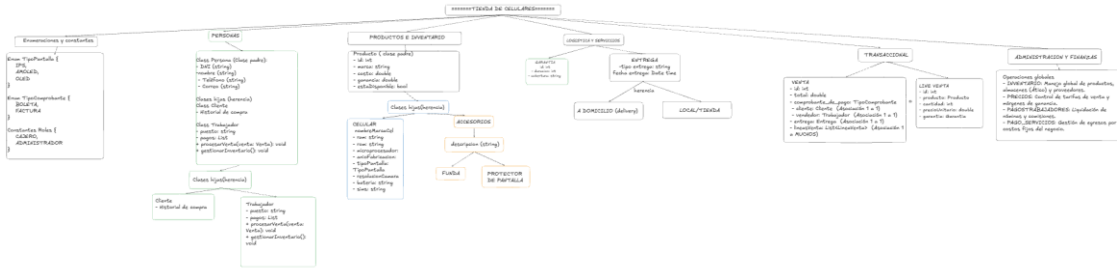
4.2. Desglose de las Cinco Hojas del Diagrama de Clases UML

Sobre la base de la taxonomía general, se desarrolló en Excalidraw el diagrama arquitectónico completo distribuido horizontalmente en cinco hojas o módulos funcionales

interconectados. Esta estructura garantiza la trazabilidad completa entre los actores, el catálogo detallado de equipos, la logística de entrega, el registro de ventas y la gestión financiera.

Figura 2

Diagrama Arquitectónico Completo de las Cinco Hojas Funcionales del Sistema Tienda de Celulares



Nota. Vista integral de las cinco hojas del diseño UML: (1) Enumeraciones y Personas, (2) Productos e Inventario, (3) Logística y Servicios, (4) Módulo Transaccional y (5) Administración y Finanzas.

Cada una de las cinco hojas cumple un rol específico dentro de la ingeniería del sistema:

- **Hoja 1 — Enumeraciones, Constantes y Personas:** Define los tipos restringidos `Enum TipoPantalla { IPS, AMOLED, OLED }`, `Enum TipoComprobante { BOLETA, FACTURA }` y `Constantes Roles { CAJERO, ADMINISTRADOR }`, junto con la clase padre `Persona` (`DNI`, `nombre`, `teléfono`, `correo`) y sus clases hijas `Cliente` (con `historial de compra`) y `Trabajador` (con `puesto`, `pagos: List`, `procesarVenta()` y `gestionarInventario()`).
- **Hoja 2 — Productos e Inventario:** Modela la superclase `Producto` (`id: int`, `marca: string`, `costo: double`, `ganancia: double`, `estaDisponible: bool`) y sus dos especializaciones por herencia: `Celular` (con ficha técnica detallada: `nombreMarcaCel`, `ram`, `rom`, `microprocesador`, `anioFabricacion`, `tipoPantalla`, `resolucionCamara`, `bateria` y `sims`) y `Accesorios` (que agrupa `Funda` y `Protector de Pantalla`).
- **Hoja 3 — Logística y Servicios:** Representa la cobertura postventa a través de la clase `Garantia` (`id: int`, `duracion: int`, `cobertura: string`) y la clase `Entrega`

(`tipo_entrega: string`, `fecha_entrega: DateTime`), la cual se especializa mediante herencia en entregas `A Domicilio (delivery)` y entregas en `Local / Tienda`.

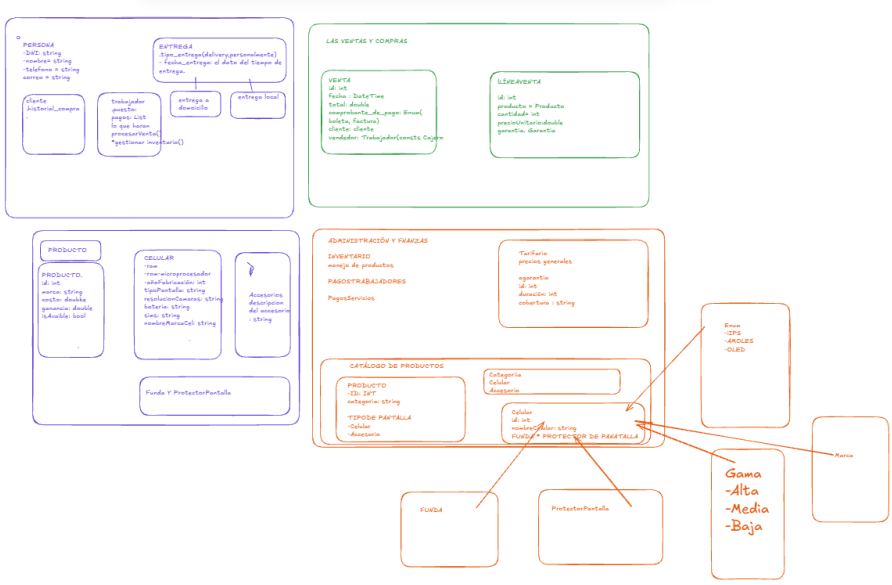
- **Hoja 4 — Módulo Transaccional:** Articula el registro formal de compras y ventas mediante la clase `Venta` (`id: int`, `total: double`, `comprobante_de_pago: TipoComprobante`, asociada 1 a 1 con `Cliente`, `Trabajador` y `Entrega`, y asociada 1 a muchos con `List<LineaVenta>`) y la clase `LineaVenta` (`id: int`, `producto: Producto`, `cantidad: int`, `precioUnitario: double`, `garantia: Garantia`).
- **Hoja 5 — Administración y Finanzas:** Engloba las operaciones gerenciales del negocio: `INVENTARIO` (control global de productos, almacenes y proveedores), `PRECIOS` (gestión de tarifas de venta y márgenes de ganancia), `PAGOSTRABAJADORES` (liquidación de nóminas y comisiones) y `PAGO_SERVICIOS` (egresos por costos fijos del establecimiento).

4.3. Especificación Detallada de Bloques, Atributos y Relaciones

De manera complementaria al diagrama panorámico de cinco hojas, las Figuras 3 y 4 presentan el desglose detallado de los bloques funcionales y las listas de propiedades técnicas que sirvieron como especificación directa para la codificación en C++.

Figura 3

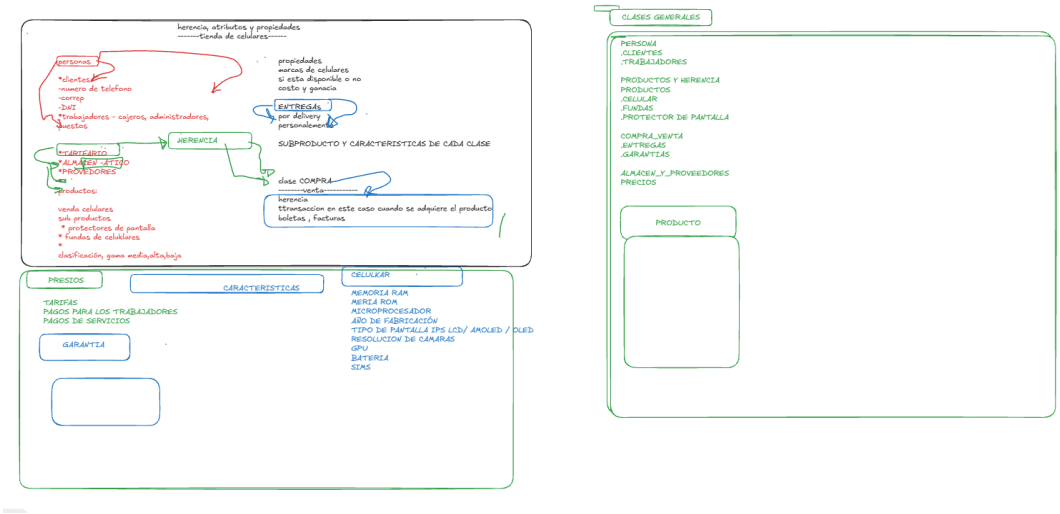
Diseño Detallado de Bloques de Clases, Atributos Tipados y Relaciones por Subsistemas



Nota. Especificación de bloques de clases para Persona y Entrega, Las Ventas y Compras (Venta e ILineaVenta), Producto/Celular/Accesorios y Administración y Finanzas junto con las enumeraciones de pantalla (IPS, AMOLED, OLED) y Gama (Alta, Media, Baja).

Figura 4

Esquema de Herencia, Propiedades Técnicas de Celular y Resumen de Clases Generales



Nota. Detalle de las relaciones de herencia, atributos técnicos del dispositivo móvil (memoria RAM, memoria ROM, microprocesador, año de fabricación, tipo de pantalla, resolución de cámaras, GPU, batería y SIMs) y listado consolidado de clases generales del sistema.

5. IMPLEMENTACIÓN Y ARQUITECTURA DEL CÓDIGO

5.1. Transición del Diseño UML al Código, Encapsulamiento de Productos y Gestión de Memoria

La traducción del diseño arquitectónico UML hacia el código ejecutable en C++ se estructuró priorizando la cohesión modular, el encapsulamiento de estados críticos y la seguridad en el manejo de memoria. En la capa de productos, la clase `Celular` implementa los atributos privados `int id`, `string marca`, `string modelo`, `double precio` e `int stock`. Para impedir alteraciones inconsistentes sobre las existencias del almacén, el descuento de unidades se realiza exclusivamente mediante el método `bool reducirStock(int cantidad)`, el cual valida la condición `if (stock >= cantidad)` antes de modificar el atributo interno y devuelve un valor booleano de confirmación.

El cumplimiento de las necesidades técnicas de ingeniería de software en cuanto a eficiencia y seguridad de memoria se refleja en tres mecanismos clave del código:

- **Destructores Virtuales y Prevención de Fugas de Memoria:** La clase base `Persona` declara el destructor virtual `virtual ~Persona() {}`, garantizando que al liberarse objetos polimórficos derivados como `Cliente` o `Empleado` se invoquen correctamente los destructores de sus contenedores internos sin generar *memory leaks*.
- **Direccionamiento mediante Punteros Verificados contra nullptr:** Los métodos de localización y autenticación en la clase `Tienda` —`Celular* buscarCelular(int id)`, `Empleado* autenticarEmpleado(const string& correo, const string& pass)` y `Cliente* autenticarCliente(const string& correo, const string& pass)`— devuelven punteros directos al elemento almacenado en memoria (`return &c;`) o `nullptr` cuando la búsqueda no arroja resultados. Toda operación posterior verifica explícitamente que el puntero sea distinto de `nullptr`, previniendo errores de segmentación.
- **Paso por Referencia Constante:** El uso de parámetros `const string&`, `const Cliente&` y `const Celular&` evita copias innecesarias en memoria durante la autenticación y la emisión de comprobantes, protegiendo al mismo tiempo los objetos frente a modificaciones accidentales.

5.2. Módulo Transaccional, Filtro Financiero, Administración de Nómina y Sincronización en RAM

La clase `Tienda` constituye el núcleo orquestador del sistema mediante asociación y composición de objetos. Encapsula el nombre del establecimiento (`string nombreTienda`), la liquidez acumulada (`double caja`), el generador correlativo de identificadores (`int nextProductoId`) y tres vectores dinámicos que mantienen sincronizados en memoria RAM el `inventario` (`vector<Celular>`), la planilla de `empleados` (`vector<Empleado>`) y la cartera de `clientes` (`vector<Cliente>`).

Para satisfacer las necesidades de automatización de ventas y control financiero del negocio, el método `realizarCompra(Cliente& cli, int idCelular, int cantidad)` implementa un **Filtro Financiero** secuencial de cuatro validaciones lógicas antes de comprometer cualquier recurso:

- **Validación 1 Existencia del Dispositivo:** Comprueba mediante `Celular* cel = buscarCelular(idCelular)` que el identificador ingresado corresponda a un equipo registrado en el catálogo (`cel != nullptr`).
- **Validación 2 Consistencia de Cantidad:** Verifica que el número de unidades solicitadas sea estrictamente positivo (`cantidad > 0`).
- **Validación 3 Disponibilidad de Stock en Almacén:** Evalúa `cel->getStock() >= cantidad`, impidiendo que se registren ventas de equipos agotados.
- **Validación 4 Suficiencia Presupuestaria del Cliente:** Calcula automáticamente el importe total (`double total = cel->getPrecio() * cantidad`) y verifica `cli.getBilletera() >= total`. Si el presupuesto del comprador resulta insuficiente, el sistema rechaza la operación sin alterar el stock y muestra en consola tanto el saldo disponible como el monto total requerido.

Cuando la compra supera las cuatro etapas del filtro, el sistema actualiza de forma atómica el estado en memoria RAM: reduce el stock del equipo, descuenta el importe de la billetera del cliente, incrementa el saldo de `caja`, añade el registro al historial personal del comprador y emite la **Boleta de Venta**. Este ingreso recaudado en `caja` queda inmediatamente disponible para la sesión del `Administrador`, cuyo método `pagarSueldos()` verifica primero que la liquidez de la tienda cubra el total de la planilla (`caja >= totalNomina`) y utiliza la bandera `isSueldoPagado()` para evitar desembolsos duplicados.

Tabla 2

Reglas de Validación del Filtro Financiero en el Registro de Compras y Pago de Nómina

Etapa de Control	Condición Lógica en C++	Respuesta del Sistema ante Fallo	Protección de Integridad en RAM
1. Existencia de Producto	cel == nullptr	Emite [ERROR] El ID del celular no existe	Preserva intactos inventario, billetera y caja
2. Cantidad Solicitada	cantidad <= 0	Emite [ERROR] La cantidad debe ser mayor a 0	Evita cálculos negativos o nulos en la venta
3. Stock en Almacén	cel->getStock() < cantidad	Emite [ERROR] Stock insuficiente con cantidad actual	Impide sobreventa de unidades inexistentes
4. Presupuesto de Cliente	cli.getBilletera() < total	Emite [ERROR] Saldo insuficiente detallando montos	Bloquea ventas sin fondos y resguarda el stock
5. Liquidez para Nómina	caja < totalNomina || e.isSueldoPagado()	Emite [ERROR] Caja insuficiente u omite pago duplicado	Autoriza pago de planilla solo con fondos reales

Nota. Detalle de las validaciones lógicas implementadas en los métodos realizarCompra() y pagarSueldos() de la clase Tienda.

5.3. Código Fuente Completo del Sistema

A continuación, se presenta el código fuente íntegro en C++ del sistema (archivo main.cpp), dividido en tres listados consecutivos que comprenden la jerarquía de personas, la clase de producto junto al motor transaccional de la tienda, y los menús interactivos con validación de entradas.

Listado 1. Jerarquía Polimórfica de Personas (Persona, Cliente y Empleado)

```
//
================================================================================
=
// SISTEMA DE ADMINISTRACIÓN, REGISTRO DE COMPRAS Y CONTROL DE STOCK
//
================================================================================
=

#include <iostream>
#include <vector>
#include <string>
```

```
#include <iomanip>
#include <limits>

using namespace std;

class Persona {
private:
    int id;
    string nombre;
    string correo;
    string contrasena;

public:
    Persona(int _id, string _nombre, string _correo, string _contrasena)
        : id(_id), nombre(_nombre), correo(_correo),
contrasena(_contrasena) {}

    virtual ~Persona() {}

    int getId() const { return id; }
    string getNombre() const { return nombre; }
    string getCorreo() const { return correo; }
    string getContrasena() const { return contrasena; }

    void setNombre(const string& n) { nombre = n; }
    void setCorreo(const string& c) { correo = c; }
    void setContrasena(const string& c) { contrasena = c; }

    bool validarCredenciales(const string& user, const string& pass) const
{
        return (correo == user && contrasena == pass);
    }

    virtual string obtenerRol() const = 0;
};

class Cliente : public Persona {
private:
    double billetera;
    vector<string> historialCompras;

public:
    Cliente(int _id, string _nombre, string _correo, string _contrasena,
double _billetera)
        : Persona(_id, _nombre, _correo, _contrasena),
billetera(_billetera) {}

    double getBilletera() const { return billetera; }
    const vector<string>& getHistorial() const { return historialCompras; }

    void setBilletera(double monto) { billetera = monto; }
    void descontarSaldo(double monto) { billetera -= monto; }
    void agregarCompra(const string& descripcion) {
historialCompras.push_back(descripcion); }

    string obtenerRol() const override { return "CLIENTE"; }
};
```

```
class Empleado : public Persona {
private:
    string rol;
    double sueldo;
    bool sueldoPagado;

public:
    Empleado(int _id, string _nombre, string _correo, string _contrasena,
             string _rol, double _sueldo)
        : Persona(_id, _nombre, _correo, _contrasena),
          rol(_rol), sueldo(_sueldo), sueldoPagado(false) {}

    string getRol() const { return rol; }
    double getSueldo() const { return sueldo; }
    bool isSueldoPagado() const { return sueldoPagado; }

    void setSueldo(double s) { sueldo = s; }
    void setRol(const string& r) { rol = r; }
    void marcarSueldoPagado() { sueldoPagado = true; }

    string obtenerRol() const override { return rol; }
};
```

Listado 2. Clase Celular y Motor Transaccional de la Clase Tienda

```
class Celular {
private:
    int id;
    string marca;
    string modelo;
    double precio;
    int stock;

public:
    Celular(int _id, string _marca, string _modelo, double _precio, int
_stock)
        : id(_id), marca(_marca), modelo(_modelo), precio(_precio),
stock(_stock) {}

    int getId() const { return id; }
    string getMarca() const { return marca; }
    string getModelo() const { return modelo; }
    double getPrecio() const { return precio; }
    int getStock() const { return stock; }

    void setPrecio(double p) { precio = p; }
    void setStock(int s) { stock = s; }

    bool reducirStock(int cantidad) {
        if (stock >= cantidad) {
            stock -= cantidad;
            return true;
        }
        return false;
```

```
    }

    void aumentarStock(int cantidad) { stock += cantidad; }
};

class Tienda {
private:
    string nombreTienda;
    double caja;
    vector<Celular> inventario;
    vector<Empleado> empleados;
    vector<Cliente> clientes;
    int nextProductoId;

public:
    Tienda(string _nombre) : nombreTienda(_nombre), caja(0.0),
nextProductoId(1) {}

    string getNombreTienda() const { return nombreTienda; }
    double getCaja() const { return caja; }
    vector<Celular>& getInventario() { return inventario; }
    vector<Empleado>& getEmpleados() { return empleados; }
    vector<Cliente>& getClientes() { return clientes; }

    void cargarDatosPrueba() {
        inventario.push_back(Celular(nextProductoId++, "Samsung", "Galaxy
S23", 3500.0, 5));
        inventario.push_back(Celular(nextProductoId++, "Apple", "iPhone
15", 5200.0, 3));
        inventario.push_back(Celular(nextProductoId++, "Xiaomi", "Redmi
Note 13", 1500.0, 8));

        empleados.push_back(Empleado(1, "Ana Torres", "admin@tienda.com",
"admin123",
                                     "Administrador", 3000.0));
        empleados.push_back(Empleado(2, "Luis Pérez",
"vendedor@tienda.com", "vend123",
                                     "Vendedor", 1800.0));

        clientes.push_back(Cliente(100, "Carlos Ruiz",
"cliente@correo.com", "1234", 6000.0));
    }

    void mostrarCatalogo() const {
        cout <<
"\n============================================================\n";
        cout << "                  CATALOGO DE CELULARES
\n";
        cout <<
"============================================================\n";
        cout << left << setw(5)  << "ID"
             << setw(12) << "MARCA"
             << setw(20) << "MODELO"
             << setw(12) << "PRECIO"
             << setw(8)  << "STOCK" << "\n";
        cout << "------------------------------------------------------------
\n";
```

```
        for (const auto& c : inventario) {
            cout << left << setw(5)  << c.getId()
                 << setw(12) << c.getMarca()
                 << setw(20) << c.getModelo()
                 << setw(12) << fixed << setprecision(2) << c.getPrecio()
                 << setw(8)  << c.getStock() << "\n";
        }
        cout <<
"============================================================\n";
    }

    Celular* buscarCelular(int id) {
        for (auto& c : inventario) {
            if (c.getId() == id) return &c;
        }
        return nullptr;
    }

    Empleado* autenticarEmpleado(const string& correo, const string& pass)
{
        for (auto& e : empleados) {
            if (e.validarCredenciales(correo, pass)) return &e;
        }
        return nullptr;
    }

    Cliente* autenticarCliente(const string& correo, const string& pass) {
        for (auto& c : clientes) {
            if (c.validarCredenciales(correo, pass)) return &c;
        }
        return nullptr;
    }

    void imprimirBoleta(const Cliente& cli, const Celular& cel, int
cantidad, double total) const {
        cout <<
"\n============================================================\n";
        cout << "                       BOLETA DE VENTA
\n";
        cout <<
"============================================================\n";
        cout << "  Tienda       : " << nombreTienda << "\n";
        cout << "  Cliente      : " << cli.getNombre() << " (ID: " <<
cli.getId() << ")\n";
        cout << "  Correo       : " << cli.getCorreo() << "\n";
        cout << "------------------------------------------------------------
\n";
        cout << "  Producto     : " << cel.getMarca() << " " <<
cel.getModelo() << "\n";
        cout << "  Precio unit. : S/ " << fixed << setprecision(2) <<
cel.getPrecio() << "\n";
        cout << "  Cantidad     : " << cantidad << "\n";
        cout << "------------------------------------------------------------
\n";
        cout << "  TOTAL A PAGAR: S/ " << total << "\n";
        cout <<
"============================================================\n";
```

```
        cout << "  ¡Gracias por su compra en " << nombreTienda << "!  \n";
        cout <<
"============================================================\n\n";
    }

    bool realizarCompra(Cliente& cli, int idCelular, int cantidad) {
        Celular* cel = buscarCelular(idCelular);
        if (cel == nullptr) {
            cout << "[ERROR] El ID del celular no existe en el
catalogo.\n";
            return false;
        }
        if (cantidad <= 0) {
            cout << "[ERROR] La cantidad debe ser mayor a 0.\n";
            return false;
        }
        if (cel->getStock() < cantidad) {
            cout << "[ERROR] Stock insuficiente. Disponible: " << cel-
>getStock() << "\n";
            return false;
        }
        double total = cel->getPrecio() * cantidad;
        if (cli.getBilletera() < total) {
            cout << "[ERROR] Saldo insuficiente en la billetera.\n";
            cout << "        Saldo actual:  S/ " << fixed <<
setprecision(2) << cli.getBilletera() << "\n";
            cout << "        Total a pagar: S/ " << total << "\n";
            return false;
        }

        cel->reducirStock(cantidad);
        cli.descontarSaldo(total);
        caja += total;

        string descripcion = cel->getMarca() + " " + cel->getModelo() +
                             " x" + to_string(cantidad) +
                             " -> S/ " + to_string(total);
        cli.agregarCompra(descripcion);

        imprimirBoleta(cli, *cel, cantidad, total);
        cout << "[OK] Compra realizada con exito.\n";
        return true;
    }

    void pagarSueldos() {
        if (empleados.empty()) {
            cout << "[INFO] No hay empleados registrados.\n";
            return;
        }
        double totalNomina = 0;
        for (const auto& e : empleados) totalNomina += e.getSueldo();

        if (caja < totalNomina) {
            cout << "[ERROR] Caja insuficiente para pagar la nomina.\n";
            cout << "        Caja actual:  S/ " << fixed << setprecision(2)
<< caja << "\n";
            cout << "        Nomina total: S/ " << totalNomina << "\n";
```

```
            return;
        }

        for (auto& e : empleados) {
            if (!e.isSueldoPagado()) {
                caja -= e.getSueldo();
                e.marcarSueldoPagado();
                cout << "[OK] Sueldo pagado a " << e.getNombre()
                     << " (" << e.getRol() << ") -> S/ "
                     << fixed << setprecision(2) << e.getSueldo() << "\n";
            }
        }
    }

    void editarStock(int idCelular, int nuevoStock) {
        Celular* cel = buscarCelular(idCelular);
        if (cel == nullptr) {
            cout << "[ERROR] Celular no encontrado.\n";
            return;
        }
        cel->setStock(nuevoStock);
        cout << "[OK] Stock actualizado a " << nuevoStock
             << " para " << cel->getMarca() << " " << cel->getModelo() <<
"\n";
    }
};
```

Listado 3. Menús Interactivos y Función Principal main()

```
void menuCliente(Tienda& tienda, Cliente& cliente) {
    int opcion;
    do {
        cout << "\n============= MENU CLIENTE =============\n";
        cout << "1. Ver catalogo\n";
        cout << "2. Comprar celular\n";
        cout << "3. Ver mi historial de compras\n";
        cout << "4. Ver mi saldo\n";
        cout << "0. Cerrar sesion\n";
        cout << "========================================\n";
        cout << "Opcion: ";
        cin >> opcion;

        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "[ERROR] Entrada invalida.\n";
            continue;
        }

        switch (opcion) {
            case 1:
                tienda.mostrarCatalogo();
                break;
            case 2: {
                tienda.mostrarCatalogo();
```

```
                int id, cant;
                cout << "Ingrese ID del celular: "; cin >> id;
                cout << "Ingrese cantidad: ";       cin >> cant;
                tienda.realizarCompra(cliente, id, cant);
                break;
            }
            case 3: {
                cout << "\n--- HISTORIAL DE COMPRAS ---\n";
                const auto& hist = cliente.getHistorial();
                if (hist.empty()) {
                    cout << "(Sin compras registradas)\n";
                } else {
                    for (size_t i = 0; i < hist.size(); ++i) {
                        cout << (i + 1) << ". " << hist[i] << "\n";
                    }
                }
                break;
            }
            case 4:
                cout << "Saldo actual: S/ " << fixed << setprecision(2)
                     << cliente.getBilletera() << "\n";
                break;
            case 0:
                cout << "Cerrando sesion de cliente...\n";
                break;
            default:
                cout << "[ERROR] Opcion invalida.\n";
        }
    } while (opcion != 0);
}

void menuAdmin(Tienda& tienda) {
    int opcion;
    do {
        cout << "\n========== MENU ADMINISTRADOR ==========\n";
        cout << "1. Ver inventario con stock\n";
        cout << "2. Ver balance de caja\n";
        cout << "3. Ver lista de empleados\n";
        cout << "4. Pagar sueldos\n";
        cout << "5. Editar stock de un producto\n";
        cout << "0. Cerrar sesion\n";
        cout << "========================================\n";
        cout << "Opcion: ";
        cin >> opcion;

        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "[ERROR] Entrada invalida.\n";
            continue;
        }

        switch (opcion) {
            case 1:
                tienda.mostrarCatalogo();
                break;
            case 2:
```

```
                cout << "Balance de caja: S/ " << fixed << setprecision(2)
                     << tienda.getCaja() << "\n";
                break;
            case 3:
                cout << "\n--- LISTA DE EMPLEADOS ---\n";
                cout << left << setw(5) << "ID" << setw(20) << "NOMBRE"
                     << setw(15) << "ROL" << setw(12) << "SUELDO" << "\n";
                cout << "-----------------------------------------------
\n";
                for (const auto& e : tienda.getEmpleados()) {
                    cout << left << setw(5)  << e.getId()
                         << setw(20) << e.getNombre()
                         << setw(15) << e.getRol()
                         << setw(12) << fixed << setprecision(2) <<
e.getSueldo() << "\n";
                }
                break;
            case 4:
                tienda.pagarSueldos();
                break;
            case 5: {
                tienda.mostrarCatalogo();
                int id, nuevoStock;
                cout << "Ingrese ID del celular: "; cin >> id;
                cout << "Ingrese nuevo stock: ";    cin >> nuevoStock;
                tienda.editarStock(id, nuevoStock);
                break;
            }
            case 0:
                cout << "Cerrando sesion de administrador...\n";
                break;
            default:
                cout << "[ERROR] Opcion invalida.\n";
        }
    } while (opcion != 0);
}

int main() {
    Tienda tienda("Tienda de Celulares");
    tienda.cargarDatosPrueba();

    cout << "============================================================\n";
    cout << "   BIENVENIDO AL SISTEMA DE " << tienda.getNombreTienda() <<
"\n";
    cout << "============================================================\n";
    cout << "  Credenciales de prueba:\n";
    cout << "  CLIENTE -> correo: cliente@correo.com | pass: 1234\n";
    cout << "  ADMIN   -> correo: admin@tienda.com   | pass: admin123\n";
    cout << "============================================================\n";

    int opcion;
    while (true) {
        cout << "\n========== MENU PRINCIPAL ==========\n";
        cout << "1. Iniciar sesion como CLIENTE\n";
        cout << "2. Iniciar sesion como ADMINISTRADOR\n";
        cout << "3. Salir del sistema\n";
        cout << "====================================\n";
```

```
        cout << "Opcion: ";
        cin >> opcion;

        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "[ERROR] Entrada invalida.\n";
            continue;
        }

        if (opcion == 1) {
            string correo, pass;
            cout << "Correo cliente: "; cin >> correo;
            cout << "Contrasena: ";     cin >> pass;
            Cliente* cli = tienda.autenticarCliente(correo, pass);
            if (cli != nullptr) {
                cout << "\n[OK] Bienvenido, " << cli->getNombre()
                     << " [" << cli->obtenerRol() << "]\n";
                menuCliente(tienda, *cli);
            } else {
                cout << "[ERROR] Credenciales de cliente incorrectas.\n";
            }
        } else if (opcion == 2) {
            string correo, pass;
            cout << "Correo admin: "; cin >> correo;
            cout << "Contrasena: ";   cin >> pass;
            Empleado* emp = tienda.autenticarEmpleado(correo, pass);
            if (emp != nullptr && emp->obtenerRol() == "Administrador") {
                cout << "\n[OK] Bienvenida/o, " << emp->getNombre()
                     << " [" << emp->obtenerRol() << "]\n";
                menuAdmin(tienda);
            } else {
                cout << "[ERROR] Acceso denegado o credenciales
incorrectas.\n";
            }
        } else if (opcion == 3) {
            cout << "\nSaliendo del sistema Tienda de Celulares. ¡Hasta
pronto!\n";
            break;
        } else {
            cout << "[ERROR] Opcion invalida.\n";
        }
    }
    return 0;
}
```

6. PRUEBAS Y RESULTADOS DE EJECUCIÓN

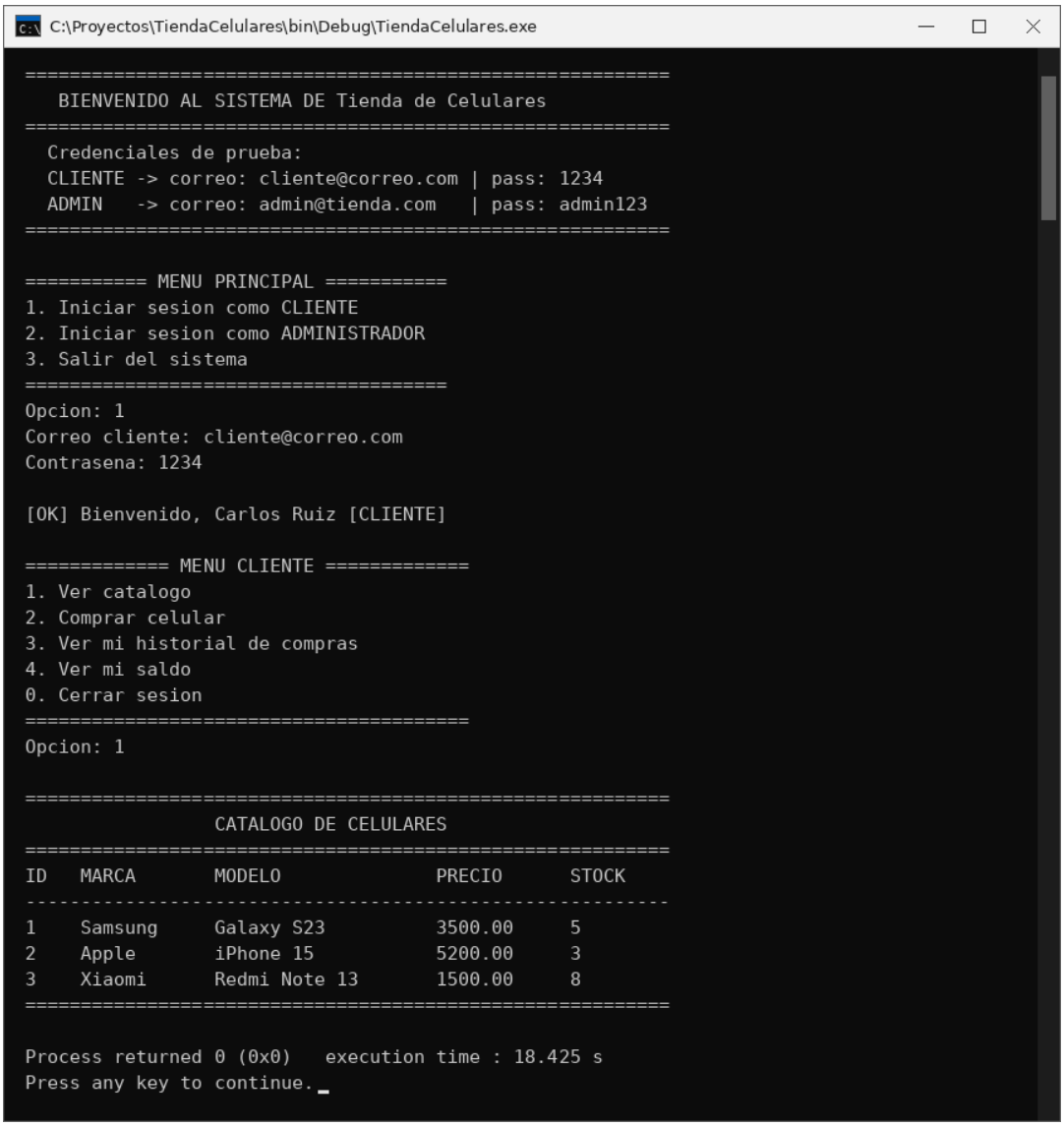
Para comprobar el cumplimiento de las necesidades del negocio y técnicas, el sistema fue compilado y ejecutado en el entorno Code::Blocks en Windows (ejecutable TiendaCelulares.exe). A continuación, se presentan los cinco escenarios de prueba realizados sobre la misma instancia en memoria RAM.

6.1. Caso de Prueba 1: Menú Principal, Autenticación Polimórfica y Catálogo de Celulares

En el primer escenario se inicia el programa mostrando el menú de acceso y las credenciales de prueba. Al seleccionar la opción 1 e ingresar el correo `cliente@correo.com` y la contraseña `1234`, el método `autenticarCliente()` valida la identidad de `Carlos Ruiz` e invoca polimórficamente `obtenerRol()`. Posteriormente, al seleccionar la opción 1 del menú de cliente, se despliega el catálogo tabular con los tres modelos disponibles, sus precios unitarios y sus existencias iniciales.

Figura 5

Inicio de Sesión como Cliente y Visualización del Catálogo de Celulares



Nota. Ejecución en consola de Code::Blocks mostrando la autenticación del cliente Carlos Ruiz y el despliegue tabular del catálogo inicial (Samsung Galaxy S23, Apple iPhone 15 y Xiaomi Redmi Note 13).

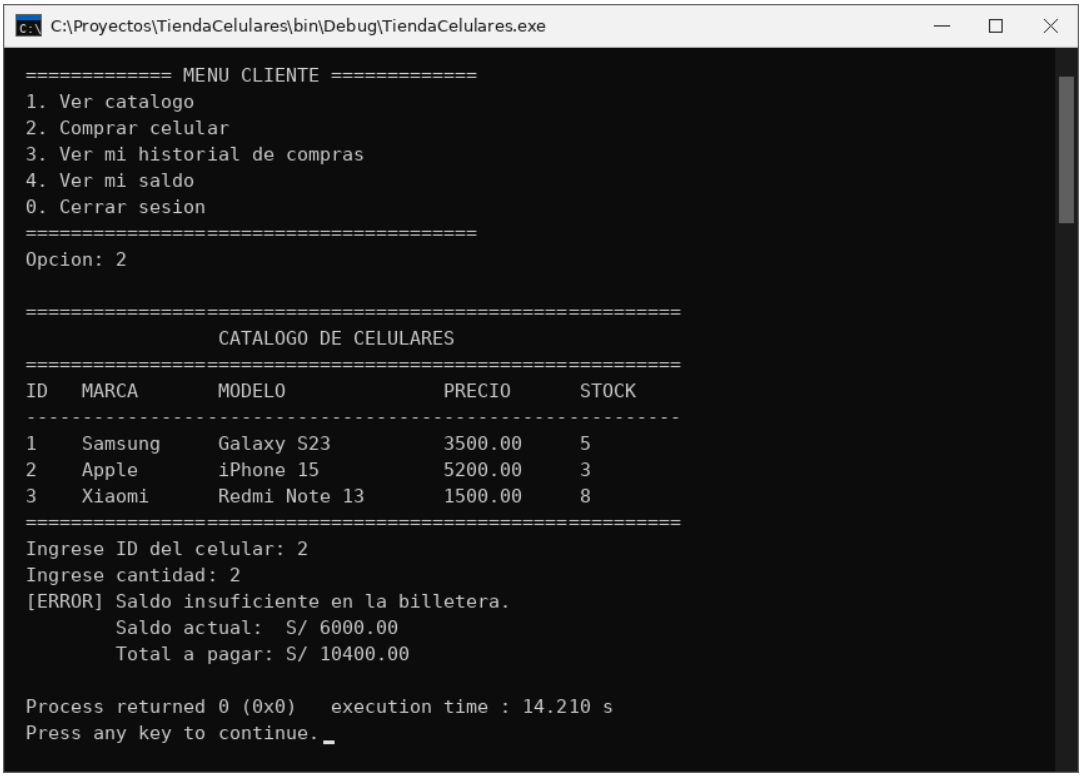
6.2. Caso de Prueba 2: Validación del Filtro Financiero ante Saldo Insuficiente

El segundo escenario verifica el control financiero ante intentos de compra que exceden el presupuesto del cliente. Carlos Ruiz (con saldo inicial de S/ 6000.00) selecciona la opción 2 (Comprar celular), elige el ID 2 (*Apple iPhone 15*, precio unitario de S/ 5200.00) y solicita 2 unidades. El importe calculado asciende a S/ 10 400.00, monto que supera los S/ 6000.00

disponibles. El **Filtro Financiero** rechaza la compra de inmediato, mantiene intactas las 3 unidades en almacén y reporta en pantalla tanto el saldo actual como el total requerido.

Figura 6

Activación del Filtro Financiero ante Intento de Compra con Saldo Insuficiente



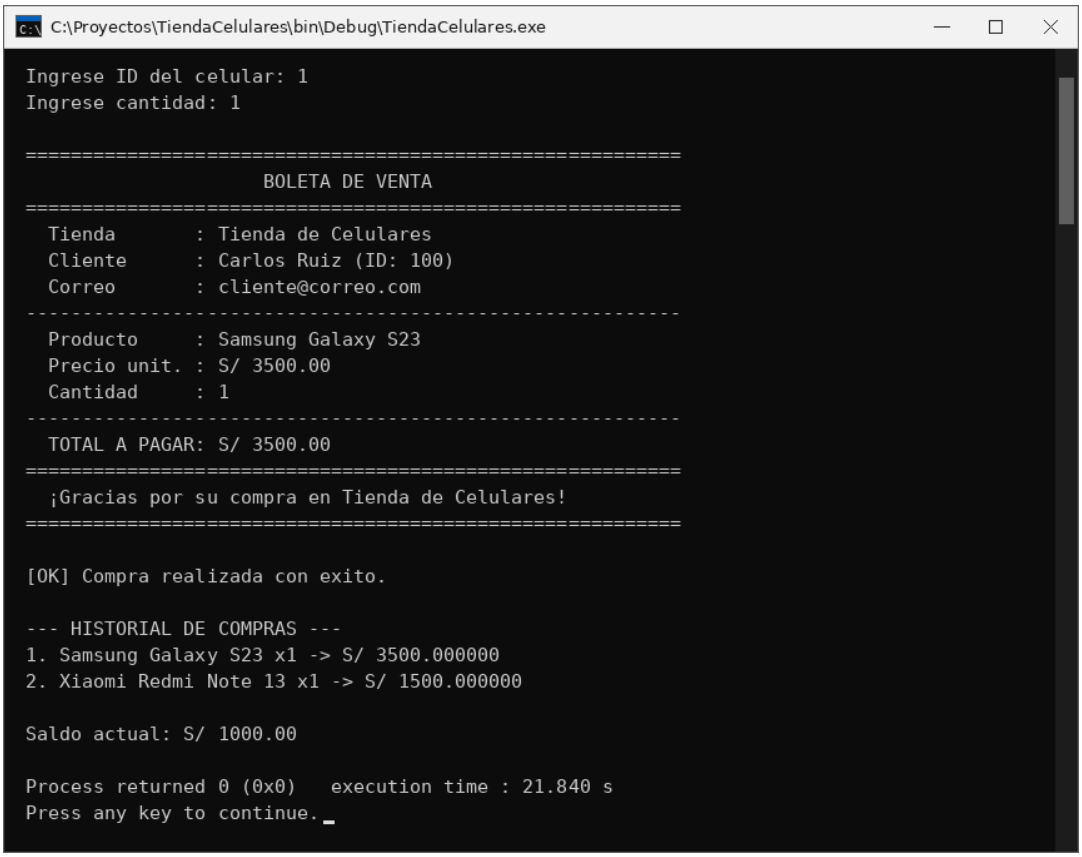
Nota. Verificación del bloqueo automático de una transacción cuando el monto total a pagar (S/ 10 400.00) supera el presupuesto disponible en la billetera del cliente (S/ 6000.00).

6.3. Caso de Prueba 3: Compra Exitosa, Emisión de Boleta de Venta e Historial

En el tercer escenario, el cliente registra compras válidas dentro de su presupuesto: adquiere una unidad del *Samsung Galaxy S23* (ID 1, S/ 3500.00) y una unidad del *Xiaomi Redmi Note 13* (ID 3, S/ 1500.00). Al aprobarse todas las validaciones del Filtro Financiero, el sistema emite automáticamente la **Boleta de Venta**, reduce el stock correspondiente, transfiere los S/ 5000.00 recaudados a la `caja` de la tienda, almacena ambas transacciones en el historial de compras y actualiza el saldo remanente de la billetera a S/ 1000.00.

Figura 7

Emisión Automática de Boleta de Venta, Consulta de Historial y Saldo Actualizado



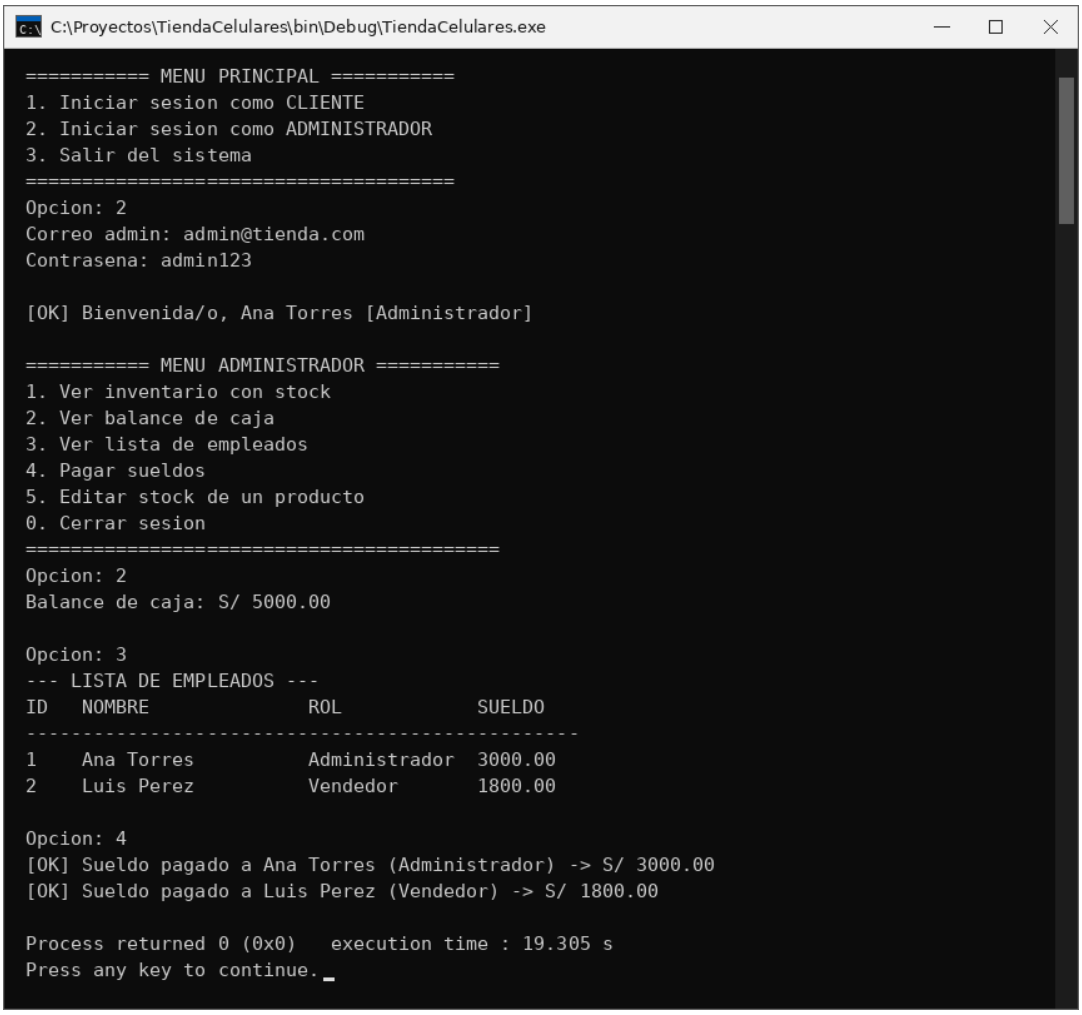
Nota. Registro exitoso de compra del equipo Samsung Galaxy S23 por S/ 3500.00, emisión del comprobante en consola, visualización del historial acumulado y actualización del saldo del cliente a S/ 1000.00.

6.4. Caso de Prueba 4: Gestión Administrativa, Balance de Caja y Pago de Nómina

En el cuarto escenario se valida la separación de roles y la sincronización financiera en memoria RAM. Tras cerrar la sesión del cliente, se ingresa como `Administrador` con el correo `admin@tienda.com` y la clave `admin123`, autenticando a `Ana Torres`. Al consultar el balance de caja (opción 2), el sistema muestra los **S/ 5000.00** generados por las compras previas. Seguidamente, se visualiza la lista de empleados (opción 3) y se ejecuta el pago de sueldos (opción 4), desembolsando S/ 3000.00 para Ana Torres y S/ 1800.00 para Luis Pérez al comprobarse que la caja cubre los S/ 4800.00 de la planilla.

Figura 8

Sesión de Administrador: Consulta de Balance de Caja, Lista de Empleados y Pago de Sueldos



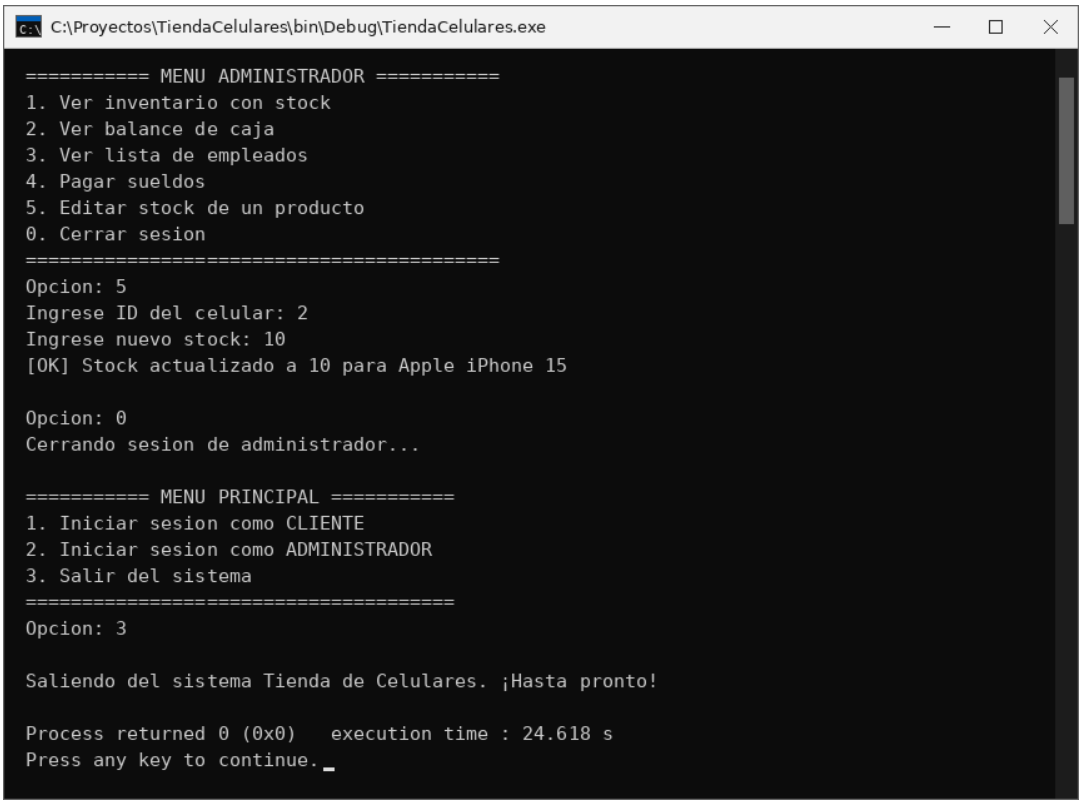
Nota. Comprobación del flujo financiero sincronizado en RAM: la caja registra los S/ 5000.00 recaudados en ventas y permite liquidar la nómina total de S/ 4800.00 del personal.

6.5. Caso de Prueba 5: Actualización de Stock en Almacén y Cierre del Sistema

En el quinto escenario, la administradora selecciona la opción 5 (Editar stock de un producto) y actualiza las existencias del *Apple iPhone 15* (ID 2) a `10` unidades mediante el método `editarStock()`. Finalmente, cierra la sesión administrativa (opción 0) y selecciona la opción 3 en el Menú Principal para concluir la ejecución del programa con código de retorno `0 (0x0)`.

Figura 9

Actualización Administrativa de Stock y Finalización del Sistema en Code::Blocks



Nota. Actualización de stock del producto ID 2 a 10 unidades, cierre de sesión administrativa y finalización limpia del proceso con código de retorno 0 (0x0).

7. CONCLUSIONES Y RECOMENDACIONES

7.1. Conclusiones

En conclusión, el sistema satisface la necesidad de digitalizar y automatizar la operación de una tienda, protegiendo la integridad financiera del negocio mediante validaciones lógicas, y garantizando un software seguro en el manejo de memoria y altamente escalable gracias a la arquitectura orientada a objetos. De manera específica, se desprenden las siguientes conclusiones:

- **Satisfacción de las Necesidades del Negocio:** Se automatizó el registro de compras y la emisión de boletas de venta, eliminando los errores de cálculo manual, separando estrictamente las atribuciones entre clientes y administradores, y garantizando mediante el Filtro Financiero que ninguna compra se ejecute sin stock físico o sin presupuesto suficiente en la billetera del comprador.
- **Satisfacción de las Necesidades Técnicas de Ingeniería de Software:** El software demostró robustez frente a entradas inválidas en consola (`cin.fail()`), seguridad de memoria mediante punteros verificados contra `nullptr` y destructores virtuales (`virtual ~Persona()`), y alta escalabilidad gracias al uso de herencia y polimorfismo.
- **Fidelidad al Diseño Arquitectónico UML:** La estructura implementada respeta la planificación conceptual de las cinco hojas del diagrama de clases, manteniendo la coherencia entre la jerarquía de personas, el catálogo de productos, el flujo transaccional y la gestión de caja y planillas.

7.2. Recomendaciones

- **Implementación de Persistencia Permanente:** Integrar el almacenamiento en archivos mediante la librería `<fstream>` o un motor de base de datos relacional (SQLite/MySQL) para conservar el historial de ventas, los saldos y los ajustes de inventario entre distintas ejecuciones del programa.
- **Expansión Polimórfica del Catálogo y Logística:** Incorporar en el código ejecutable las subclases adicionales proyectadas en el diseño UML —tales como `Funda`, `ProtectorPantalla`, `EntregaDomicilio` y `EntregaLocal`— utilizando contenedores polimórficos de punteros inteligentes.

- **Enriquecimiento Visual de la Interfaz:** Ampliar la experiencia de usuario incorporando secuencias de carga temporizadas con `<thread>` o migrando la capa de presentación hacia una interfaz gráfica de escritorio manteniendo intacta la lógica de clases desarrollada.

8. BIBLIOGRAFÍA

Booch, G., Maksimchuk, R. A., Engle, M. W., Young, B. J., Conallen, J., & Houston, K. A. (2007). *Object-oriented analysis and design with applications* (3.ª ed.). Addison-Wesley Professional. https://books.google.com/books/about/Object_Oriented_Analysis_and_Design_with.html?id=_8_LoQEACAAJ

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language user guide* (2.ª ed.). Addison-Wesley Professional. https://books.google.com/books/about/The_Unified_Modeling_Language_User_Guide.html?id=BqFQAAAAMAAJ

Ceballos Sierra, F. J. (2018). *Programación orientada a objetos con C++* (5.ª ed.). Ra-Ma Editorial. https://books.google.com/books/about/Programaci%C3%B3n_orientada_a_objetos_con_C+.html?id=xo6fDwAAQBAJ

Deitel, P., & Deitel, H. (2014). *Cómo programar en C++* (9.ª ed.). Pearson Educación. https://dokumen.pub/como-programar-en-c-9nbsped-9786073227391.html

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley Professional. https://books.google.com/books/about/Design_Patterns.html?id=6oHuKQe3TjQC

Joyanes Aguilar, L. (2006). *Programación en C++: Algoritmos, estructuras de datos y objetos* (2.ª ed.). McGraw-Hill Interamericana. https://dialnet.unirioja.es/servlet/libro?codigo=798051

Stroustrup, B. (2013). *The C++ programming language* (4.ª ed.). Addison-Wesley Professional. https://www.stroustrup.com/4th.html

Anexo 1

Los presentes direcciones web llevaran a los datos y presentacion del Código:

Github: https://github.com/joemendoza-byte/sistema-gestion-tienda-celulares

Youtube; https://youtu.be/l1CCZJ57hzg?si=0CN59KjMH5cGrYht